

Appunti 02/03/26 – Sicurezza – Bella copia

Sicurezza nelle Web Application e Struttura delle Richieste HTTP

In una web application, non tutte le funzionalità sono accessibili indistintamente a qualsiasi utente. Alcune sezioni dell'applicazione sono riservate esclusivamente a chi possiede determinati ruoli all'interno del sistema. Il concetto fondamentale su cui si basa questa distinzione è quello di endpoint: un endpoint è un indirizzo esposto verso l'esterno che risponde a una chiamata HTTP, e un'API non è altro che un insieme di tali endpoint. In un'applicazione moderna, gestire la sicurezza significa essenzialmente stabilire chi ha il diritto di accedere a ciascun endpoint. La sede reale della sicurezza risiede nel backend, poiché è lì che vengono applicate le regole di controllo degli accessi. Di conseguenza, la maggior parte degli endpoint viene resa privata, rendendola inaccessibile senza le opportune autorizzazioni.

Per comprendere come funziona questo meccanismo è necessario analizzare la struttura di una richiesta HTTP. Ogni richiesta è composta da due parti distinte: una testa e un corpo. La testa raccoglie gli header, ovvero le intestazioni, che hanno il compito di fornire al server informazioni aggiuntive e di contesto utili a interpretare correttamente la richiesta. Il corpo, invece, contiene il dato vero e proprio trasmesso, che nella maggior parte dei casi si presenta in formato JSON. Esistono due header particolarmente rilevanti: **Content-Type**, che specifica il formato del contenuto presente nel corpo della richiesta, e **Accept**, che comunica al server in quale formato il client si aspetta di ricevere la risposta. Nel caso di una richiesta di tipo GET come quella mostrata negli appunti, il corpo è assente e tutte le informazioni sono contenute esclusivamente nell'header.

La versione del protocollo utilizzata è HTTP/1.1, introdotta nel 1997 e ancora oggi ampiamente in uso. Questa versione ha introdotto le connessioni persistenti, i metodi PUT, PATCH, DELETE e OPTIONS, e lo scambio di dati in formato testuale.

Una richiesta HTTP è quindi identificabile attraverso tre componenti principali: il verbo HTTP, che indica il tipo di operazione da eseguire, l'URL, che rappresenta la risorsa su cui operare, e gli header, che forniscono il contesto necessario all'elaborazione. Nell'esempio riportato, il client — rappresentato da Postman — invia una richiesta GET al server in ascolto su `localhost:8080`. Il server elabora la richiesta e restituisce una risposta che include il corpo con i dati, lo status code **200 OK** a conferma del successo dell'operazione, e i metadati relativi ai tempi di risposta, pari a 66 ms per un payload di 1.24 KB.

Autenticazione, JWT e Observable in Angular

Una volta che un endpoint viene reso privato, non è più sufficiente inviare una semplice richiesta HTTP per ottenere una risposta: è necessario contestualmente dimostrare la propria identità. Il flusso prevede che, prima di poter eseguire qualsiasi chiamata GET verso una risorsa protetta, il client debba effettuare una richiesta POST all'endpoint di login per autenticarsi.

Inviare le credenziali, ovvero email e password, ad ogni singola richiesta sarebbe una soluzione tecnicamente impraticabile, poiché comporterebbe un accesso al database ad ogni chiamata, con un impatto significativo sulle prestazioni. La filosofia alla base di questo sistema è che il server non ha interesse a mantenere memoria di tutti i propri client: è il client stesso ad avere la responsabilità di dimostrare chi è. Questo avviene attraverso il JWT, un token che viene rilasciato dal server al momento del login con risposta **200 OK** e che il client deve conservare e allegare a tutte le richieste successive. La differenza rispetto a una chiamata non autenticata risiede nella presenza

dell'header `Authorization: Bearer <token>`, dove il valore del token corrisponde esattamente al JWT ricevuto durante il login. Senza di esso il server risponde con `403 Forbidden`, rifiutando l'accesso alla risorsa.

Sul versante del codice Angular, il metodo `this.http.get<any>()` dell'`HttpClient` non restituisce direttamente i dati della risposta, bensì un oggetto di tipo `Observable<any>`. Un `Observable`, proveniente dalla libreria RxJS, rappresenta un'operazione asincrona che emetterà il proprio risultato in un momento futuro, in modo concettualmente simile a una promessa ma con capacità più avanzate. La caratteristica fondamentale di un `Observable` è che non esegue alcuna operazione finché qualcuno non vi si sottoscrive esplicitamente. Per questo motivo, subito dopo la chiamata `.get()`, viene invocato il metodo `.subscribe()`, all'interno del quale si definiscono due callback: `next`, che viene eseguita quando la risposta arriva con successo ricevendo i dati come argomento, ed `error`, che viene eseguita in caso di fallimento della richiesta, ricevendo l'oggetto di errore. Questo meccanismo è funzionalmente analogo ai blocchi `.then()` e `.catch()` della `fetch` nativa di JavaScript.

Struttura del Token JWT e Decodifica lato Client

Il metodo `generateToken` costruisce il payload del token utilizzando una `HashMap<String, Object>`, una struttura a coppie chiave-valore che permette di inserire dinamicamente informazioni di natura eterogenea. La scelta di `String` come tipo di chiave risponde alla specifica del protocollo JWT, che richiede identificatori univoci per ciascun campo del payload, come `"id"`, `"role"` o `"email"`. La scelta di `Object` come tipo di valore è invece necessaria per consentire la coesistenza di dati di tipo diverso all'interno della stessa struttura: numeri interi per gli identificativi, stringhe per i nomi, e potenzialmente strutture più complesse. All'interno della mappa vengono inseriti i dati anagrafici e tecnici dell'utente, ovvero identificativo, nome, cognome, username, ruolo ed email. La struttura è inoltre flessibile in senso condizionale: se l'utente è associato a una struttura alberghiera, la mappa viene estesa dinamicamente per includere anche i dati dell'hotel, come codice, nome, indirizzo e città, senza dover prevedere una struttura rigida per ogni tipologia di utente. Una volta popolata, la mappa viene passata al costruttore del token tramite `setClaims`, diventando il payload ufficiale del JWT. La validità temporale è impostata su dieci ore dall'emissione. La libreria `Jwts` converte automaticamente la mappa in formato JSON, serializzando ciascun valore in base al suo tipo originale, e sigilla il tutto con una firma digitale basata sull'algoritmo HS256 e sulla chiave segreta del server. Ogni server possiede una propria chiave, che può essere generata all'avvio o aggiornata automaticamente.

La verifica dell'integrità del token avviene ad ogni richiesta: quando il client invia il token, il server ricalcola la firma a partire dall'header e dal payload ricevuti, usando la propria chiave segreta. Se anche un solo carattere del payload fosse stato modificato, il risultato del ricalcolo sarebbe diverso dalla firma presente nel token, e il server rifiuterebbe immediatamente la richiesta riconoscendo che i dati sono stati alterati.

Sul lato client, il codice TypeScript/Angular all'interno del blocco `next` del `.subscribe()` gestisce la risposta del login nel seguente modo. Dalla risposta del server viene estratto il token tramite `(json as any).token`, dove `as any` istruisce TypeScript a sospendere i controlli di tipo in quel momento. Il token viene poi suddiviso nelle sue tre componenti separando la stringa sul carattere punto tramite `split(".")`, ottenendo un array di tre elementi corrispondenti a

intestazione, corpo e firma. Si isola quindi la seconda parte, ovvero il payload, che a questo stadio è ancora codificata in Base64. La funzione nativa del browser `atob()` decodifica quella stringa Base64 restituendo il testo JSON in chiaro. Il nome `atob` sta per "ASCII to Binary" nel senso di decodifica Base64: prende una stringa come `"SGVsbG8="` e la riconverte nel testo originale `"Hello"`. La funzione inversa è `btoa`, che percorre il tragitto opposto codificando una stringa normale in Base64. Infine, `JSON.parse()` converte la stringa JSON decodificata in un oggetto JavaScript a tutti gli effetti, rendendo accessibili i singoli campi come `payload.role` o `payload.sub`. La callback `error` è invece preposta alla gestione dei fallimenti della chiamata HTTP, limitandosi a stampare l'errore in console.